Pretty straightforward stuff, `'hello'` occurs three times, as does `'ll'`, and there are six instances of `l` in `$sayHello`. `substr_count()` is also binary safe:

```
var_dump(substr_count("a\x00 a\x00 a\x00", "\x00"));
var_dump(substr_count("a\x00 a\x00 a\x00", 'a'));
```

```
Output:
int(3)
int(3)
```

You can also supply offset and length arguments to test sub-string of the first argument:

```
var_dump(substr_count($sayHello, "hello", 0, 11));
```

```
Output:
int(2)
```

When it comes to multi-byte safety, it really comes down to knowing what encoding you're dealing with. There is an `mbstring` function `mb_substr_count()`, which will analyze the encodings of both input strings (and internally convert them to UTF-8 if either is in a different format); otherwise, it functions identically to `substr_count()`. Take a look at the following:

```
$name = "\u{C7}a\u{11F}ari";
var_dump(substr_count($name, 'a'));
var_dump(substr_count($name, "\u{C7}"));
```

```
Output:
int(2)
int(1)
```

As we can see this is working correctly, because `substr_count()` is simply traversing the bytes of `$name` and looking for the same byte sequences in `'a'` (the first call) or `"\u{C7}"` (the second call). The encoding (which should be UTF-8 if following coding standard PSR-1) used to create the string data for `$name` will match the encoding for the string literals, so the byte sequences will match, and we'll get a correct count. If we change the encoding of `$name` using the `mb_convert_encoding()` function to UTF-16, we get the following: